12-DETERMINISM-CONTRACT.md — Same input, same bytes, every time

> 🎯 **Rule R3 — Same input, same bytes.** Identical RAW + identical recipe + identical model versions must produce a bit-identical output on the same execution provider, and a ΔE2000 ≤ 0.5 output across providers. Determinism is not a nicety here — it is what makes golden tests, regression detection and "Explain My Edit" honest.

1. Why this file exists

Without a determinism contract you cannot tell the difference between *"the new denoiser is worse"* and *"the GPU scheduled tiles in a different order today"*. Every quality gate in `18-MODEL-EVAL-GATES.md` and every golden diff in `05-QA-STRATEGY.md` is meaningless unless this file is obeyed.

2. The three determinism levels

Level	Guarantee	Applies to	Verified by
D0 — Bit-exact	Byte-identical output file	Same machine, same EP, same model hashes, same build	SHA-256 of the rendered TIFF
D1 — Perceptually exact	ΔE2000 max ≤ 0.5, mean ≤ 0.1; no pixel differs ≥ 2/255	Across CUDA / DirectML / CoreML / CPU	Cross-EP parity job

Level	Guarantee	Applies to	Verified by
D2 — Decision-stable	Identical cull set, identical star ranks, identical scene labels	Across runs and machines	Decision-ledger diff

D2 is the one photographers actually feel: re-running autopilot must not deliver a different 800 photos.

3. Sources of non-determinism and their mandated fixes

Source	Symptom	Mandated fix	Owner
Unordered parallel reduction (`par_iter().sum()`)	Float sum differs run to run	Deterministic reduction: fixed chunking + ordered merge, or Kahan/pairwise sum	SRC
`HashMap` iteration order	Different tie-break in ranking	`BTreeMap` or `IndexMap` anywhere output depends on order	TLC
Unseeded RNG (dither, augmentation, sampling)	Noise pattern changes	All RNG derives from `run_seed ⊕ photo_id ⊕ stage_id`; no thread-local entropy	SRG
Wall-clock time or locale in logic	Different rounding, different sort	Clock is an injected trait; locale forced to `C` for all numeric formatting	SRC
GPU tile scheduling / atomics	Non-associative float accumulation	No atomic float accumulation in shaders; fixed tile order; two-pass reduce	SRG
Fast-math / FMA contraction differences	Last-bit drift across EPs	Disable fast-math; pin `-ffp-contract=off`; document as D1 not D0	SRG

Source	Symptom	Mandated fix	Owner
cuDNN / TensorRT autotuning	Different kernel each session	Deterministic algo selection; cache and pin the chosen tactic per model hash	MLOPS
Model quantisation nondeterminism	Score flips near a threshold	Fixed calibration set; store quantisation params in the model package	SRML
Threshold ties in culling	Two photos swap on re-run	Total ordering: score → capture time → filename → photo UUID	MLL
Cloud LLM sampling	Different story segmentation	`temperature = 0`, `top_p = 1`, pinned model version, response cached by prompt hash	AGT
Filesystem enumeration order	Import order changes burst grouping	Sort by (capture time, camera serial, filename) before any processing	SRC
Build-embedded paths / timestamps	Binary differs per build	`SOURCE_DATE_EPOCH`, `--remap-path-prefix`	DEVOPS

4. The Render Fingerprint (frozen contract)

Every rendered output carries a fingerprint. If two fingerprints match, the bytes must match — that is the whole test.

```
// crates/aura-render/src/fingerprint.rs — FROZEN CONTRACT v1
#[derive(Debug, Clone, PartialEq, Eq, Hash, serde::Serialize, serde::Deserialize)]
pub struct RenderFingerprint {
    pub schema: u16,                 // 1
    pub source_raw_sha256: String,
    pub recipe_canonical_sha256: String, // canonical JSON, sorted keys, fixed float fmt
    pub pipeline_version: String,    // semver of the develop graph
```

```
    pub shader_bundle_sha256: String,
    pub model_set_sha256: String,    // sorted "name@version:
sha" joined by '\n'
    pub colour_pipeline: String,     // "ProPhotoLinear->Disp
lay-P3"
    pub execution_provider: String,  // "cuda", "directml",
"coreml", "cpu"
    pub output_profile: String,      // "sRGB IEC61966-2.1"
    pub run_seed: u64,
}

impl RenderFingerprint {
    /// Stable 32-hex digest written into the export sidecar
and the ledger.
    pub fn digest(&self) -> String { /* blake3 of canonical C
BOR */ }
}
```

Canonical recipe JSON rules (frozen): keys sorted lexicographically · no insignificant whitespace · floats serialised with `{:.6}` and `-0.0` normalised to `0.0` · arrays never reordered · absent ≠ null · UTF-8 NFC.

5. Seed derivation (frozen)

```
run_seed        = blake3(project_uuid || run_index)
// stored in the run row
photo_seed      = blake3(run_seed || photo_uuid)
stage_seed(s)   = blake3(photo_seed || stage_name || stage_ve
rsion)
```

No component may call a global RNG. `rand::thread_rng()` is a banned symbol in `clippy.toml`.

6. Cross-EP parity policy

- Every model in `02-AI-MODEL-STACK.md` ships with a parity report: CPU reference vs each accelerated EP on the 200-image parity corpus.
- Gate: **max abs logit delta ≤ 1e-3**, **decision flip rate ≤ 0.1 %**, **ΔE2000 max ≤ 0.5** for image-producing models.
- A model that fails parity on an EP is disabled on that EP and falls back — it never ships silently degraded. Emits `AURA-ML-5012 EpParityFailed` at `Degraded`.

7. Cloud calls and determinism

Cloud output is inherently non-deterministic, so it is **contained**:

1. All cloud responses are validated against a JSON Schema and **cached** keyed by `blake3(prompt_template_version || input_digest || model_id)`.
2. The cache is part of the project, so re-running a wedding offline reproduces the same decisions — satisfying D2.
3. Cloud output may only *propose*; a local deterministic rule applies it. A cloud hiccup can therefore never change pixels unpredictably.
4. If the cache is missing and the network is unavailable, the local fallback path runs and is recorded as `Degraded`, with the ledger noting the decision came from the fallback.

8. Golden corpus and hash manifest

```
tests/golden/
  manifest.toml            # photo -> expected fingerprint di
gest + expected SHA-256
  wedding-a-indoor/        # 40 frames, tungsten + daylight m
ix
  wedding-b-outdoor/       # 40 frames, harsh sun + backlight
  wedding-c-disaster/      # 30 frames, corrupt, mixed camera
s, extreme ISO
  parity-200/              # cross-EP parity corpus
```

CI job `golden-determinism`:

1. Render the corpus twice in the same job → assert byte equality (catches intra-run nondeterminism).
2. Render on Linux CPU and compare to `manifest.toml` → assert D0.
3. Render on the Windows CUDA runner and the macOS CoreML runner → assert D1 tolerances.
4. Diff the decision ledger against `manifest.toml` → assert D2.

Any intentional change requires regenerating the manifest **in its own commit**, with before/after contact sheets attached to the PR and sign-off from `QAIQ` + `COL`.

9. Test plan

- **Repeat test** — same input rendered 5× in-process and 5× across process restarts; all 10 digests equal.
- **Thread-count sweep** — render with 1, 2, 8, 32 worker threads; digests must be identical.
- **Shuffle test** — shuffle input file order; final gallery, cull set and ranks must be identical.
- **Seed test** — changing `run_seed` must change dither noise but must **not** change any cull decision or star rank.
- **Clock test** — run with the injected clock set to three different dates; outputs identical.
- **Locale test** — run under `tr_TR.UTF-8` (the classic dotted-I / decimal-comma trap); outputs identical.

10. Acceptance criteria

- [ ] `RenderFingerprint` implemented exactly as frozen and written to every export sidecar.
- [ ] `golden-determinism` CI job green on all three OS runners.
- [ ] Thread-count sweep and shuffle test pass.

- [ ] `rand::thread_rng`, `HashMap` iteration in output paths, and `SystemTime::now()` in logic are all lint-banned.
- [ ] Every shipped model has a parity report stored in `models/<name>/parity.json`.
- [ ] The determinism section of the run report shows the fingerprint for every delivered file.

11. Claude Code prompt

> Act as `SRG` (Senior Engineer — GPU). Implement `RenderFingerprint` exactly as frozen in `12-DETERMINISM-CONTRACT.md`, including canonical recipe serialisation and the seed derivation chain. Audit the existing develop pipeline for every source of non-determinism listed in section 3 and produce a table of file:line → issue → fix before changing code. Then switch hats to `QAL` and implement the repeat, thread-sweep, shuffle, clock and locale tests. Report each as PASS/FAIL with command output. Do not regenerate any golden manifest without asking me first.